

BUILD A SERVERLESS DATA LAKE ON AWS

By balraj kular | June 2026

The screenshot displays the Amazon Athena Query Editor interface. On the left sidebar, the 'Data source' is set to 'AwsDataCatalog', 'Catalog' is 'None', and 'Database' is 'datalake_db'. Under 'Tables and views', there are two tables listed: 'processed' and 'raw'. The main query editor area contains the following SQL code:

```
1 -- Verify revenue = quantity * unit_price
2 SELECT order_id, quantity, unit_price, revenue,
3        ROUND(CAST(quantity AS DOUBLE) * unit_price, 2) AS expected_revenue
4 FROM processed
5 WHERE ROUND(CAST(quantity AS DOUBLE) * unit_price, 2) != revenue
6 LIMIT 10;
```

Below the query editor, the 'Query results' tab is active, showing a status bar with 'Completed', 'Time in queue: 99 ms', 'Run time: 445 ms', and 'Data scanned: 23.75 KB'. The results table is empty, with columns: '#', 'order_id', 'quantity', 'unit_price', 'revenue', and 'expected_revenue'. The status 'No results' is displayed at the bottom.

Project Overview: Serverless Data Lake on AWS

What this project builds and why it matters

In this project, I'm building a serverless data lake pipeline on AWS that ingests raw e-commerce order data, cleans it with automated quality checks, and makes it queryable with SQL. Raw data is messy—it arrives with missing values, duplicates, and no structure. Before anyone can run reports or spot trends, someone must build the pipeline to clean and organize it.

I will create S3 storage zones (raw and processed), use AWS Glue Crawlers and the Data Catalog to automatically discover schemas, write a Python Shell ETL job to remove nulls, deduplicate records, and calculate revenue, then query the cleaned data with Amazon Athena.

By the end, I'll have a fully functional serverless pipeline where raw data flows in, gets cataloged, transformed with quality checks, and delivered as clean, analytics-ready data—without managing any servers. This is the exact pattern companies use for business intelligence and machine learning.

Configuring the AWS Environment and Data Zones

Setting up S3 buckets and IAM permissions

In this step, I'm setting up the foundational AWS infrastructure for my data lake pipeline. I'll create two S3 buckets—one to store the actual data lake files (raw and processed zones) and another to hold Athena query results separately for cleaner organization. Then I'll create an IAM role with a trust policy that allows AWS Glue to assume it, and attach both the AWS-managed Glue service policy and a custom inline policy granting full S3 access to my buckets. Finally, I'll generate 500+ rows of realistic e-commerce order data with intentional quality issues—null values and duplicate rows—using a Python script, and upload the CSV to the raw zone of my data lake. This foundation ensures my pipeline has storage, proper permissions, and messy data ready for transformation in the next steps.

IAM role and policy configuration

I created an IAM role called `GlueServiceRole` that allows AWS Glue to assume it and access my S3 buckets. This role has two main permissions attached:

AWS-managed policy (`AWSGlueServiceRole`) – Grants Glue baseline permissions to run crawlers and jobs, including writing to CloudWatch Logs, EC2 networking, and accessing Glue Data Catalog resources.

Custom inline policy (`S3AccessPolicy`) – Grants full S3 access (`s3:*`) specifically to my two buckets: `serverless-project-bucket-balraj` (data lake) and `athena-results-bucket-balraj` (Athena query results). This includes read, write, and delete permissions on all objects within these buckets.

The trust policy allows the Glue service (`glue.amazonaws.com`) to assume this role, so when my crawlers and ETL jobs run, they inherit these permissions automatically. This setup follows the principle of least privilege—my Glue resources can only access the buckets they need, and nothing else.

```
balrajcular@Balrajs-MacBook-Pro aws-project % aws s3 cp orders_raw.csv s3://serverless-project-bucket-balraj/raw/orders_raw.csv
upload: ./orders_raw.csv to s3://serverless-project-bucket-balraj/raw/orders_raw.csv
balrajcular@Balrajs-MacBook-Pro aws-project % aws s3 ls s3://serverless-project-bucket-balraj/raw/
```

Discovering Schema Automatically with AWS Glue Crawler

Cataloging raw data with a Glue Crawler

In this step, I'm making my raw data discoverable by AWS analytics services. I have 520 rows of CSV sitting in my S3 bucket, but no service knows what columns or data types that file contains. Before any analytics service can query my data, it needs a schema. I'll create a Glue Data Catalog database called `datalake_db` to organize my table metadata, then create a Glue Crawler named `raw-orders-crawler` that points to my raw data prefix. When I run the crawler, it automatically scans my CSV file, infers column names and data types by sampling the data, and registers the results in the Data Catalog. Finally, I'll verify the inferred schema using `aws glue get-table` to confirm columns like `order_id` (bigint), `customer_id` (string), `quantity` (string), and `unit_price` (double) were correctly detected. This catalog entry is what enables Athena to query my raw data later.

```
{
  "Table": {
    "Name": "raw",
    "DatabaseName": "datalake_db",
    "Owner": "owner",
    "CreateTime": "2020-06-10T19:33:43-04:00",
    "UpdateTime": "2020-06-10T19:33:43-04:00",
    "LastAccessTime": "2020-06-10T19:33:43-04:00",
    "Retention": 0,
    "StorageDescriptor": {
      "Columns": [
        {
          "Name": "order_id",
          "Type": "bigint"
        },
        {
          "Name": "customer_id",
          "Type": "bigint"
        },
        {
          "Name": "product_name",
          "Type": "string"
        },
        {
          "Name": "quantity",
          "Type": "bigint"
        },
        {
          "Name": "unit_price",
          "Type": "double"
        },
        {
          "Name": "order_date",
          "Type": "string"
        },
        {
          "Name": "status",
          "Type": "string"
        }
      ],
      "Location": "s3://serverless-project-bucket-balraj/raw/",
      "InputFormat": "org.apache.hadoop.mapred.TextInputFormat",
      "OutputFormat": "org.apache.hadoop.hive ql.io.HiveIgnoreKeyTextOutputFormat",
      "Compressed": false,
      "NumberOfBuckets": -1,
      "SerdeInfo": {
        "SerializationLibrary": "org.apache.hadoop.hive.serde2.lazy.LazySimpleSerDe",
        "Parameters": {
          "field.delim": ","
        }
      }
    },
    "BucketColumns": [],
    "SortColumns": [],
    "Parameters": {
      "skip.header.line.count": "1",
      "sizekey": "26265",
      "objectcount": "1",
      "UPDATED_BY_CRAWLER": "raw-orders-crawler",
      "recordcount": "442",
      "CrawlerSchemaSerializerVersion": "1.0",
      "averageRecordSize": "55",
      "compressionType": "none"
    }
  }
}
```

Understanding inferred data types

The crawler inferred `order_id` as `bigint`, `quantity` as `bigint`, and `unit_price` as `double` from my raw CSV data.

`order_id` was inferred as `bigint` because every value in that column is a whole number with no nulls or missing values, so the crawler safely identified it as a numeric integer type. `quantity` was inferred as `bigint` as well, which means the null values I intentionally introduced in about 5% of rows either weren't sampled or were still recognized as numbers, allowing the crawler to detect a consistent numeric pattern. `unit_price` was inferred as `double` because the column contains decimal values (e.g., 9.99, 499.99), so a floating-point type was needed to preserve the precision of the prices.

The crawler samples the data and selects the safest type based on what it finds. Whole numbers become integers, decimals become doubles, and text becomes strings. This ensures Athena and other analytics services can query the data correctly for calculations or text operations.

Building and Running the Python Shell ETL Job

Designing the ETL transformation pipeline

In this step, I'm building the ETL pipeline that cleans the raw data. Raw data almost always has quality problems—missing values, duplicate records, and missing derived fields. I'll write a Python script that reads the raw CSV from S3, applies data quality checks to remove nulls in critical columns and deduplicate records, calculates a derived revenue column ($\text{quantity} \times \text{unit_price}$), and writes the cleaned output to a processed zone in my data lake. Then I'll deploy this script to AWS Glue as a Python Shell job using the `--max-capacity 0.0625` flag for cost efficiency, and run the job to transform my data. Finally, I'll verify the cleaned output exists in my processed S3 prefix. This transforms messy raw data into a reliable, analytics-ready dataset that business users can trust.

Data quality checks: null removal, deduplication, and derived columns

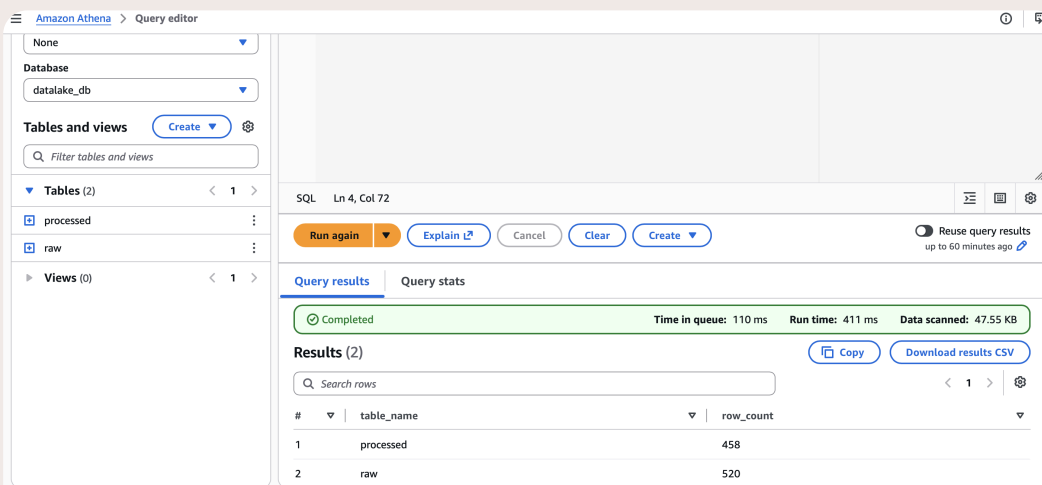
The ETL script performs four key data quality checks. First, it removes rows with null values in critical columns (`order_id`, `customer_id`, `quantity`) using `dropna()`, ensuring no incomplete records remain. Second, it deduplicates records by keeping only the first occurrence of each `order_id` with `drop_duplicates()`, eliminating the ~20 duplicate rows. Third, it enforces correct data types by casting `order_id`, `customer_id`, and `quantity` to integers and `unit_price` to float. Fourth, it calculates a derived revenue column as `quantity * unit_price`, rounded to 2 decimals. These checks transform messy raw data into clean, reliable, analytics-ready records.

```
balraj@balrajs-MacBook-Pro:~/aws-project % aws s3 ls s3://serverless-project-bucket-balraj/processed/
2026-06-18 20:02:39          24322 orders_cleaned.csv
```

Validating Data Quality with Amazon Athena

Querying processed data with serverless SQL

In this step, I'm validating that my ETL pipeline actually works. My cleaned data is sitting in the processed/ prefix, but Athena can't query it yet because it needs a catalog entry. I'll create a second crawler targeting the processed/ prefix to discover the schema of my cleaned data, including the new revenue column. Once cataloged, I'll run validation queries in Athena to confirm my pipeline delivered quality data—checking for zero nulls in critical columns, zero duplicate order_ids, and 100% accurate revenue calculations. I'll also run analytics queries like total revenue by product and average order value, then compare row counts between raw and processed tables to prove my pipeline removed the nulls and duplicates. This confirms my data is clean, reliable, and ready for business intelligence.



The screenshot displays the Amazon Athena Query Editor interface. On the left sidebar, the 'Database' is set to 'datalake_db' and 'Tables and views' are listed, including 'processed' and 'raw'. The main panel shows a query execution summary with a green 'Completed' status bar. Below this, the 'Query results' tab is active, displaying a table with 2 rows and 3 columns: '#', 'table_name', and 'row_count'.

#	table_name	row_count
1	processed	458
2	raw	520

Quantifying pipeline data quality improvements

My ETL pipeline removed 62 rows from the original dataset, reducing it from 520 rows in the raw table to 458 rows in the processed table.

Null values in critical columns - My pipeline removed rows where critical fields like `order_id`, `customer_id`, or `quantity` were missing. I intentionally introduced nulls in about 5% of rows to simulate real-world data quality issues, and the `dropna()` function eliminated these incomplete records.

Duplicate `order_ids` - I introduced approximately 20 duplicate rows to simulate data ingestion errors, and my pipeline used `drop_duplicates(subset=["order_id"], keep="first")` to keep only the first occurrence of each unique `order_id`.

The 62-row difference represents the data quality improvement my pipeline delivers. Raw data contained nulls and duplicates that would cause errors or inaccurate analytics if queried directly. By cleaning this data, my pipeline ensures analysts and downstream systems work with complete, unique, and reliable records.

Reflections and Key Takeaways

Tools and concepts mastered

This project gave me hands-on experience with AWS serverless data engineering tools and fundamental data pipeline concepts. I learned to use S3 as a data lake with raw and processed zones, Glue Crawlers for automated schema discovery, and the Glue Data Catalog as a central metadata repository. I built a Python Shell ETL job in Glue using pandas for data transformations—removing nulls, deduplicating records, and calculating derived columns—while understanding cost optimization with 0.0625 DPU over Spark. I also learned to query data with Amazon Athena using standard SQL, validate pipeline quality with aggregation queries, and compare raw vs processed row counts to prove data quality improvements. This end-to-end experience taught me how raw data flows through discovery, transformation, and analytics—exactly what companies need for business intelligence and machine learning.

Time investment and challenges

The project took approximately 55 minutes to complete from start to finish. Breaking it down: setting up S3 buckets and IAM roles (~10 mins), generating sample data and uploading to S3 (~10 mins), creating and running Glue Crawlers (~15 mins, including wait time), writing and deploying the Python Shell ETL job (~10 mins), and running Athena validation queries (~10 mins). The longest wait was for crawlers to complete (about 2 minutes each due to Glue's minimum billing period). Overall, it was a well-paced project suitable for completing within an hour.

I did this project today to learn how to build a complete serverless data lake pipeline on AWS using S3, Glue, and Athena. I wanted hands-on experience with automated schema discovery using Glue Crawlers, building cost-effective ETL jobs with Python Shell, and validating data quality with SQL queries in Athena. I also wanted to understand how raw data flows through discovery, transformation, and analytics to deliver clean, reliable datasets for business intelligence.

Another skill I want to learn is CI/CD pipeline automation using GitHub Actions or Jenkins to automatically build, test, and deploy data pipelines whenever code changes are pushed. I'm also interested in exploring orchestration tools like Apache Airflow for scheduling and monitoring complex ETL workflows in production environments.